

# Patrones

**1\_** Utilice el patrón “Adapter” para convertir la interfaz de una clase externa en una interfaz que el cliente espera, haciendo que la interfaz actúe como traductor.

**2\_** Utilice “Template Method” ya que los empleados tenían comportamientos en común, con este patrón maximizo la reutilización de código y evitó código repetido.

**3\_** Use el patrón “Adapter” para integrar la clase “VideoStream”, creando la clase “VideoStreamAdapter” que tiene una variable de “VideoStream”, es una extensión de la clase “Media” y sabe responder al mensaje “Play()”

**4\_** Use el patrón “Composite” para tratar de la misma forma a las topografías simples(agua, tierra, pantano) y a las composiciones de objetos(mixta). También apliqué Factory Method para simplificar la creación de las clases y solo tener 2 tipos de topografías.

**5\_** Use Factory Method para la creación de las sustancias químicas, porque siempre utilizan la misma fórmula para su creación.

**6\_** Use el patrón Builder para la creación de sándwich, porque todos siguen la misma serie de pasos para su creación.

**7\_** Use Factory Method para desacoplar la creación de la funcionalidad de los productos, en este caso el cliente solo pide el producto que necesita, sin importar cómo se construye. La única desventaja de este patrón es la complejidad estructural que presenta.

**8\_** Use el patrón state para representar los cambios de estado y evitar condicionales.

**9\_** Use el patrón strategy porque te da la posibilidad de cambiar los algoritmos en tiempo de ejecución.

**10\_** Use el patrón state para representar los estados de la calculadora y que estos sepan responder a los mensajes correspondientes. También uso Template Method para organizar los estados de cálculo y evitar código repetido.

**11\_** Use el patrón composite para tratar de la misma forma a los archivos y directorios.

**12\_** Use el patrón strategy para representar las distintas políticas de cancelación. Ventajas de diseño:

- **Bajo acoplamiento:** El auto no sabe cómo se calcula el reembolso, solo sabe que alguien lo hace por él.
- **OCP(open/close principle):** El diseño es abierto para extensión, se pueden agregar nuevas políticas de cancelación, sin necesidad de modificar el funcionamiento del programa.